Basics Day 1 — Session 1 (Hours 1–4)

Python Programming (Basic) • Orientation through Numbers & Operators

Session 1 Overview

Topics Covered Today

- Hour 1: Orientation + environment readiness
- Hour 2: First scripts: print(), comments, and reading errors
- Hour 3: Variables + basic types
- Hour 4: Numbers + operators

Hour 1: Orientation + Environment Readiness

Learning Outcomes

- Access the lab/VM and run Python
- Create a workspace folder and run a first script
- Understand the lab environment and file persistence

Platform Quick Start

LogicalCHOICE → LogicalLabs

- LogicalLabs opens in a **new browser tab**
- Allow pop-ups for the LogicalCHOICE domain
- **Two-key reminder**: Course content key ≠ lab key
  - Lab Access Key must be redeemed from **Current Training → Redeem Training Key**

Save and Suspend Culture

Best Practices

- **Save All** before breaks
- If disconnected, relaunch from the tile
- Files persist even if session state resets

Workspace Standard

All course work goes in a **single course folder**

- Create `/data` subfolder later for data files
- Avoids path/import issues

Course Workflow

```
Lecture → Demo → Lab
```

How Python Executes

- **Interpreter mode**: interactive testing
- **Script file**: save and run programs

Demo: Environment Setup

Steps

1. Launch LogicalLabs from LogicalCHOICE course tile
2. Open terminal / IDE in the lab VM
3. Run: `python --version`
4. Create folder: `python_basics/`
5. Create `hello_course.py` and run it
6. Save file, close it, re-open to confirm location

Lab: Environment Check + First Run

Time: 25-35 minutes

Tasks

- Create a folder for the course
- Create `hello_course.py` that prints your name and today's date
- Run it from terminal AND from the IDE

Completion Criteria

✓ Script runs without errors from both terminal and IDE
✓ Learner can find the output and locate the file on disk
✓ Learner can explain where course folder is located
✓ Learner confirms files persist after relaunch

Common Pitfalls (Hour 1)

⚠️ Wrong interpreter selected in IDE
⚠️ Saving file outside workspace
⚠️ Confusing terminal path vs file path

Optional Extensions

- Add a second script that imports `datetime` and prints formatted date
- Add a 'run instructions' comment header

Quick Check

Question: What's the difference between running in the REPL and running a `.py` file?

Hour 2: First Scripts — print(), Comments, and Reading Errors

Learning Outcomes

- Use `print()` to produce output
- Explain indentation and syntax rules at a basic level
- Read and fix common beginner errors

The print() Function

Basic Usage

```
print("Hello, world!")
print("Welcome to Python Basics")
```

Multiple Arguments

```
print("Hello", "Python", "learners")
# Output: Hello Python learners
```

Comments in Python

Single-line Comments

```python
# This is a comment
print("This is code")  # Comment after code
```

Why We Comment

- **Explain "why"**, not "what"
- Add headers/run instructions
- Document tricky logic

Python Style Standards (Basics)

Key Rules

- **4-space indentation** (no tabs)
- **Descriptive names** (snake_case)
- **Prefer f-strings** for readable output
- Comments explain "why," not "what"

```python
# Good
user_name = "Alice"
total_count = 42

# Avoid
x = "Alice"
tc = 42
```

Common Beginner Errors

Missing Quotes

```
# ❌ Error
print(Hello)

# ✓ Fixed
print("Hello")
```

Missing Parentheses

```
# ❌ Error
print "Hello"

# ✓ Fixed
print("Hello")
```

Reading Error Messages

Example Error

```
Traceback (most recent call last):
  File "script.py", line 3
    print(Hello)
          ^^^^^
NameError: name 'Hello' is not defined
```

What to Look For

1. **File name** and **line number**
2. **Error type** (NameError, SyntaxError, etc.)
3. **Error message** description

Demo: 5 Common Mistakes

Live-code these errors and fix them:

1. Missing closing quote
2. Missing closing parenthesis
3. Typo in function name
4. Using smart quotes (from copy/paste)
5. Wrong indentation

Goal: Fix each in < 30 seconds

Lab: Greeter

Time: 25-35 minutes

Tasks

- Write a program that prints a 3-line greeting
- Add a comment describing what the program does
- Intentionally create a syntax error, read the message, then fix it

Completion Criteria

✓ Output prints 3 lines correctly
✓ Learner can describe what changed when fixing the error

Common Pitfalls (Hour 2)

⚠️ Using smart quotes from copy/paste

⚠️ Missing closing parentheses/quotes

Optional Extensions

- Print a simple ASCII box around the greeting using `\n` and spacing

```
print("*" * 30)
print("*  Welcome to Python!      *")
print("*" * 30)
```

Quick Check

Question: Why is whitespace/indentation important in Python?

Hour 3: Variables + Basic Types

Learning Outcomes

- Create variables and assign values
- Use `type()` to inspect values
- Understand basic data types

What is a Variable?

Assignment

```
message = "Hello"
count = 42
price = 19.99
is_active = True
```

Reading Assignment Right to Left

```
x = 5  # 5 is assigned TO x
```

Naming Rules

Valid Identifiers

```
user_name = "Alice"
total_count = 100
max_score_2024 = 95
```

Invalid Identifiers

```
# ❌ These don't work
2nd_place = "Bob"     # starts with number
user-name = "Alice"   # uses hyphen
class = "Python"      # reserved keyword
```

Basic Data Types

Strings (str)

```
name = "Alice"
city = 'New York'
```

Numbers

```
age = 25            # int
price = 19.99       # float
```

Booleans

```
is_student = True
has_graduated = False
```

The type() Function

Checking Types

```
name = "Alice"
age = 25
price = 19.99
is_active = True

print(type(name))      # <class 'str'>
print(type(age))       # <class 'int'>
print(type(price))     # <class 'float'>
print(type(is_active)) # <class 'bool'>
```

Variable Reassignment

Changing Values

```
count = 10
print(count)  # 10

count = 20
print(count)  # 20
```

Changing Types

```
value = 42        # int
value = "Hello"   # now str
```

Demo: Profile Card

Build a small profile card with variables:

```
name = "Alice Smith"
city = "Seattle"
favorite_number = 7
likes_python = True

print("Profile Card")
print("Name:", name)
print("City:", city)
print("Favorite Number:", favorite_number)
print("Likes Python:", likes_python)
```

Lab: Profile Card

Time: 25-35 minutes

Tasks

- Store: name, city, favorite number, and a boolean (e.g., `likes_python`)
- Print a formatted 'card' showing all values
- Use `type()` for each value and print the result

Example Output

```
=== Profile Card ===
Name: Alice Smith (type: <class 'str'>)
City: Seattle (type: <class 'str'>)
Favorite Number: 7 (type: <class 'int'>)
Likes Python: True (type: <class 'bool'>)
```

Completion Criteria (Hour 3)

- ✓ Program prints values and types correctly
- ✓ Learner uses clear variable names

Common Pitfalls (Hour 3)

⚠️ Overwriting variable with a different type accidentally

⚠️ NameError from typos

⚠️ Using reserved keywords as variable names

Optional Extensions

- Add a computed value (favorite number + 10)
- Add user input for one field using `input()`

```
name = input("Enter your name: ")
print("Hello,", name)
```

Quick Check

Question: What's the difference between `'5'` and `5`?

Hour 4: Numbers + Operators

Learning Outcomes

- Use arithmetic operators and understand precedence
- Explain integer vs float results
- Build simple calculations

Arithmetic Operators

Basic Operations

```
# Addition
result = 10 + 5       # 15

# Subtraction
result = 10 - 5       # 5

# Multiplication
result = 10 * 5       # 50

# Division (always returns float)
result = 10 / 5       # 2.0
```

More Operators

Floor Division

```
result = 10 // 3      # 3 (integer division)
result = 10 / 3       # 3.333...
```

Modulo (Remainder)

```
result = 10 % 3       # 1 (remainder)
result = 10 % 2       # 0 (even number check)
```

Exponentiation

```
result = 2 ** 3       # 8 (2 to the power of 3)
```

Order of Operations

PEMDAS

```
result = 2 + 3 * 4          # 14 (not 20)
result = (2 + 3) * 4        # 20 (parentheses first)
result = 2 ** 3 * 4         # 32 (exponent first)
```

Best Practice

Use parentheses for clarity:

```
# Less clear
total = price * 1.08 + shipping

# More clear
total = (price * 1.08) + shipping
```

Integer vs Float Division

Division Always Returns Float

```
result = 10 / 2       # 2.0 (not 2)
result = 9 / 3        # 3.0 (not 3)
```

Floor Division Returns Integer

```
result = 10 // 3      # 3
result = 10 // 2      # 5
```

Rounding Numbers

The round() Function

```
value = 3.14159
rounded = round(value, 2)    # 3.14

# For display, not storage
price = 19.999
display_price = round(price, 2)  # 20.0
```

Demo: Mini Calculator

Show Division Differences

```
print("Regular division:")
print(10 / 3)        # 3.333...

print("Floor division:")
print(10 // 3)       # 3

print("Modulo (remainder):")
print(10 % 3)        # 1
```

Even/Odd Check

```
number = 7
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Lab: Tip Calculator

Time: 25-35 minutes

Tasks

- Ask for bill total and tip percent
- Compute tip amount and final total
- Display results to 2 decimals using `round()` or formatting

Example Interaction

```
Enter bill total: 50.00
Enter tip percent (e.g., 15 for 15%): 18

Tip amount: $9.00
Final total: $59.00
```

Sample Solution Structure

```python
# Get input
bill = float(input("Enter bill total: "))
tip_percent = float(input("Enter tip percent: "))

# Calculate
tip = bill * (tip_percent / 100)
total = bill + tip

# Display
print(f"Tip amount: ${tip:.2f}")
print(f"Final total: ${total:.2f}")
```

Completion Criteria (Hour 4)

- ✓ Correct totals for sample inputs
- ✓ Uses float conversion where needed
- ✓ Displays amounts with 2 decimal places

Common Pitfalls (Hour 4)

⚠️ Forgetting to convert `input()` to float

⚠️ Integer division surprises (using `/` vs `//`)

⚠️ Rounding too early in calculations

Optional Extensions

- Add split count (divide by number of people)
- Add sales tax as an extra percent
- Handle $0 bill gracefully

```
# Split bill
num_people = int(input("Split among how many people? "))
per_person = total / num_people
print(f"Per person: ${per_person:.2f}")
```

Quick Check

Question: What does the `%` (modulo) operator return and when is it useful?

Session 1 Wrap-Up

What We Covered Today

Hour 1: Orientation

- Lab environment setup
- File persistence and workspace organization

Hour 2: First Scripts

- `print()` function
- Comments and error reading

Hour 3: Variables

- Variable assignment and naming

Scope Guardrail Reminder

Stay in Basics Scope

✓ Core syntax and basic types
✓ Simple calculations and string operations
✓ Clear variable names and comments

Not Yet (Advanced Topics)

✗ List comprehensions
✗ Lambda functions
✗ Decorators
✗ Generators
✗ Advanced OOP patterns

Homework / Practice

Recommended Exercises

1. Create a program that calculates area of different shapes
2. Build a simple currency converter
3. Create a BMI calculator with formatted output
4. Practice fixing syntax errors deliberately

Next Session Preview

Session 2 (Hours 5–8)

- Hour 5: String indexing and slicing
- Hour 6: String methods (normalize, search, replace)
- Hour 7: Input/output + type conversion
- Hour 8: Checkpoint 1 (mini-assessment)

Questions?

Remember:

- Save your work frequently
- Practice reading error messages
- Use clear, descriptive variable names
- Comment your code when helpful

Thank You!

See you in Session 2!